

Movie Reservation System

Microservices Architecture

Version 2.0 | 2026
Portfolio Project — Public Repository Documentation

1. Overview

Movie Reservation System is a cinema management platform for ticket reservations, seat management, and payment processing (mock). The system is implemented as four loosely-coupled microservices communicating via REST, with asynchronous workflows handled through an event bus.

Each service owns its own database. No service reads or writes to another service's database directly. Cross-service data references are stored as opaque identifiers (integers or UUIDs), never as foreign keys across service boundaries.

1.1 Service Map

Service	Responsibility	Owns
IAM Service	Authentication, authorization, JWT issuance	User (Django AbstractUser), Role, RefreshToken
Location Service	Reference data for geography and cinemas	Country, City, Cinema
Catalog Service	Movies, scheduling, venue layout	Movie, Genre, Showtime, CinemaRoom, CinemaRoomRow, CinemaRoomSeat, TicketType
Reservation Service	Booking, payments(mock), tickets, QR/PDF generation	Reservation, ReservationTicket

1.2 Communication Patterns

- Synchronous REST — used for queries where the caller needs an immediate response (e.g. Reservation → Catalog to check seat availability)

- Asynchronous events — used for state transitions that fan out to multiple consumers (e.g. `PaymentSucceeded` → trigger ticket generation + email delivery)
- JWT — each microservice validate the JWT on every request without calling IAM again

2. Authentication & JWT Strategy

The platform uses asymmetric JWT signing (ES256 — ECDSA with P-256 curve). IAM holds the private key and is the only service that can issue tokens. Every other service holds only the public key, which it uses to verify signatures without being able to mint new tokens.

2.1 Key Distribution via Docker Volume

The public key is distributed as a read-only Docker volume mounted into each service container. This avoids a key-fetching HTTP call at startup and removes the runtime dependency on IAM for verification.

2.2 JWT Payload

Claim	Type	Description
sub	string (UUID)	User.uuid from IAM service — uuid primary key on the iam_user table
email	string	User email
role	string	User role: admin customer other
iat	integer	Issued-at timestamp
exp	integer	Expiry timestamp (Unix) — 15 min for access tokens
jti	string (UUID)	Unique token ID — used for revocation via Redis blocklist

Refresh tokens are long-lived (7 days), stored in the IAM database, and exchanged for new access tokens via `POST /auth/refresh`. On logout, the jti of the active access token is added to a Redis blocklist with a TTL matching the remaining token lifetime.

3. IAM Service

Swagger HTML Docs: <http://localhost/api/iam/admin/docs/>

Owns all identity and access management. No other service reads or writes user data. The User model extends Django AbstractUser.

3.1 Endpoints

Method	Path	Auth	Description
POST	/auth/register	Public	Create user, send verification email (log e-mail token in console if DEBUG true)
POST	/auth/verify-email	Public	Confirm email token, set status=active
POST	/auth/login	Public	Return access + refresh JWT pair
GET	/auth/refresh/access_token	Refresh token	Issue new access token
GET	/auth/refresh/refresh_token	Refresh token	Rotate refresh token
POST	/auth/logout	Access token	Revoke refresh token, add jti to Redis blocklist
GET	/auth/me	Access token	Return current user details
PATCH	/auth/me	Access token	Update first_name, last_name
GET	/admin/	Admin	Django Admin Panel

4. Location Service

Swagger HTML Docs:

Read-heavy reference data. Staff manage this through an admin interface; customers only read it.

4.1 Endpoints

Method	Path	Auth	Description
GET	/countries	Public	List all countries
GET	/countries/{code}/cities	Public	Cities in a country
GET	/cities/{id}/cinemas	Public	Cinemas in a city
GET	/cinema/{id}	Public	Single cinema detail
GET	/admin/	Staff/Admin	Django Admin Panel

5. Catalog Service

Swagger HTML Docs:

Manages the content catalogue (movies, genres), scheduling (showtimes), and physical venue layout (rooms, rows, seats). Also owns TicketType pricing, which is scoped per country using an ISO country code string — not a foreign key to the Location service.

5.1 Endpoints

Method	Path	Auth	Description
GET	/movies	Public	List movies (filterable by genre)
GET	/movies/{id}	Public	Movie detail
GET	/showtimes/schedule/{cinema_id}	Public	List showtimes (filter by date, movie).
GET	/showtimes/{id}	Public	Showtime details
GET	/showtimes/{id}/seats	Public	Available seats for a showtime (returned with ticket_types for every seat)
GET	/internal/showtime/seats	Internal	Returns details needed by other microservices (reservation process)

6. Reservation Service

Handles the full booking transaction: seat hold, payment via Stripe (mock), ticket record creation, QR code and PDF generation, and email delivery. All steps are part of one bounded context — they succeed or fail together.

This service stores `user_id` and `showtime_id` as opaque references. It calls Catalog synchronously to fetch seat availability and Stripe for payment. It never calls IAM — user identity comes from the JWT claims decoded by the API Gateway.

6.1 Seat Hold Strategy

Before a reservation is confirmed, seats are held in Redis with a TTL of 10 minutes. The hold key is stored on the Reservation record so it can be released on cancellation or expiry.

- Hold key format: `seat_hold:{showtime_id}:{seat_id}` — value: `reservation_id`
- If payment is not completed within 10 minutes, a Celery beat task expires the hold and sets reservation status to cancelled
- The hold is released immediately on successful payment or explicit cancellation

6.2 Internal Pipeline

The following steps execute in sequence on reservation creation:

- Acquire seat hold in Redis (10 min TTL per seat)
- Create Reservation record with `status=in_progress`
- Create Stripe `PaymentIntent`, return `client_secret` to frontend
- On Stripe webhook `payment_intent.succeeded`: update `payment_status=paid`, `status=confirmed`
- Generate `ReservationTicket` rows (one per seat), each with `uuid` + random secret
- Generate QR code per ticket: encode `{ uuid, secret, showtime_id }` as JSON
- Render PDF per ticket (movie, cinema, seat, showtime, customer name, QR image)
- Upload PDFs to object storage (S3 / MinIO), store path in `pdf_path`
- Send confirmation email with PDF attachments to `reservation.email`

6.3 Ticket Validation

Door scanners call `POST /tickets/validate` with the QR payload. The service looks up the ticket by `uuid`, verifies the secret matches, and checks `scanned_at` is null. If valid, it sets `scanned_at = now()` and returns 200. A grace window of 30 seconds prevents rejection on double-scans.

6.4 Endpoints

Method	Path	Auth	Description
POST	<code>/reservation</code>	Optional JWT	Begin reservation, acquire seat hold
GET	<code>/reservation/{id}</code>	Owner / Staff	Reservation detail + ticket list

Method	Path	Auth	Description
DELETE	/reservation/{id}	Owner / Staff	Cancel reservation, release hold
POST	/reservation/{id}/payment	Owner	Create Stripe PaymentIntent
POST	/webhooks/stripe	Stripe signature	Handle payment_intent.succeeded etc.
GET	/ticket/{uuid}	Owner / Staff	Single ticket + PDF download URL
POST	/ticket/validate	Staff (internal)	Validate QR at door (optional query parameter: <i>showtime</i> to check if ticket is for specific event)

7. Deployment Notes

7.1 Technology Stack

Service	Framework	Database	Cache
IAM	Django + DRF	PostgreSQL	Redis (token blocklist)
Location	FastAPI	PostgreSQL	Redis (response cache, 1hr TTL)
Catalog	FastAPI	PostgreSQL	Redis (seat availability, 1min TTL)
Reservation	FastAPI + Celery	PostgreSQL	Redis (seat holds, 10min TTL)

7.2 Cross-Service References

When a service stores a reference to a resource owned by another service, it stores only the ID — no foreign key constraint exists across databases. The receiving service is responsible for validating that the referenced ID exists before writing.

8. Change Log

Document Version	Date	Author	Notes
1.0	2026-05-19	Julian Korgol	First draft — 4-service architecture with ES256 JWT
1.1	2026-05-26	Julian Korgol	IAM Microservice changes